

On-Device Acoustic Event Detection: A Comparative Study of CNN, CRNN, and Transformer Architectures under the Accuracy–Efficiency Trade-off

Members	Rohan Sinha (230911005) • Akshdeep Singh (230911466) • Siddharth Nair (230953284)
Dataset	UrbanSound8K (8,732 clips, 10 classes, predefined 10 folds)
Submission	Interim (Phase 2), due 25 Sep 2026

1. Project Summary and Progress to Date

This project compares three deep-learning families, a convolutional network (CNN), a convolutional-recurrent network (CRNN) and an Audio Spectrogram Transformer (AST), for ten-class urban sound classification on UrbanSound8K, with the focus on the accuracy–efficiency trade-off for edge deployment. All models use the same audio source, the official 10-fold protocol, and one metric set: accuracy and macro-F1 for quality; parameters, on-disk size and CPU latency for efficiency. A compression axis (post-training quantization and pruning) is applied afterwards. This report fixes every design choice for the study so that the three members implement against one specification.

Status against the synopsis work plan:

#	Milestone (from synopsis)	Planned	Status at interim
1	Dataset acquisition, EDA, shared log-mel pipeline	05 Sep	Pipeline specified (Section 3)
2	CNN baseline + mobile-native variant	18 Sep	Design frozen (Section 4.2); preliminary results in Section 4.5
3	CRNN (CNN + BiLSTM/GRU)	22 Sep	Design frozen (Section 4.3); preliminary results in Section 4.5
4	Interim submission	25 Sep	This document
5	AST fine-tuning	12 Oct	Design frozen (Section 4.4); starts after models 1–2
6	Compression + latency/size benchmarking	20 Oct	Protocol frozen (Section 4.6)
7	Comparison, error analysis, final report	29 Oct	Pending (Section 6)

2. Literature Review

The table below summarises ten papers that frame the project: the dataset and its evaluation protocol, the CNN and CRNN lines of work, the attention-based model, the efficiency toolkit, and training techniques that we reuse. The final column states how each paper bears on our design, so the table compares the work rather than merely listing it.

#	Paper	Method	Dataset	Key result	Relevance to this project
1	Salamon, Jacoby & Bello (2014), <i>A Dataset and Taxonomy for Urban Sound Research</i> , ACM-MM	Introduces an urban-sound taxonomy and the UrbanSound8K corpus with 10 pre-sorted folds	UrbanSound8K (8,732 clips)	Establishes the 10-fold protocol that prevents leakage between clips from the same source recording	Our dataset and evaluation protocol; folds are never re-split so all models are comparable
2	Piczak (2015), <i>Environmental Sound Classification with CNNs</i> , MLSP	Shallow CNN on log-mel spectrogram segments (with delta channel), averaged over segments	ESC-50, ESC-10, UrbanSound8K	About 73% on UrbanSound8K (as reported in later comparisons); beat MFCC-based baselines	Origin of the CNN-on-log-mel recipe; our CNN baseline is a modern descendant

#	Paper	Method	Dataset	Key result	Relevance to this project
3	Salamon & Bello (2017), <i>Deep CNNs and Data Augmentation for Environmental Sound Classification</i> , IEEE SPL	SB-CNN: three conv layers with small kernels; time-stretch, pitch-shift, DRC and noise augmentation	UrbanSound8K	About 79% with augmentation, above the same CNN without augmentation and a dictionary-learning baseline	Reference CNN accuracy on this exact benchmark; motivates our augmentation choices
4	Çakır et al. (2017), <i>CRNNs for Polyphonic Sound Event Detection</i> , IEEE/ACM TASLP	Conv layers for local spectro-temporal features, then recurrent layers for temporal context	TUT-SED Synthetic 2016, TUT-SED 2009, CHiME-Home	CRNN outperformed CNN-only and RNN-only baselines on the evaluated SED sets	Template for our CRNN (CNN extractor + BiGRU)
5	Gong, Chung & Glass (2021), <i>AST: Audio Spectrogram Transformer</i> , Interspeech	Convolution-free ViT-style model; spectrogram split into patches, ImageNet-pretrained weights adapted to audio	AudioSet, ESC-50, Speech Commands V2	0.485 mAP (ensemble) on AudioSet, 95.6% on ESC-50, 98.1% on Speech Commands V2	Model 3 of our study; shows the accuracy ceiling that we then weigh against its size and latency
6	Howard et al. (2019), <i>Searching for MobileNetV3</i> , ICCV	Hardware-aware NAS plus NetAdapt, h-swish and squeeze-excite blocks	ImageNet, COCO, Cityscapes	MobileNetV3-Large is 3.2% more accurate than MobileNetV2 on ImageNet with 20% lower latency	Backbone of our mobile-native CNN variant (the 'efficient by design' point)
7	Jacob et al. (2018), <i>Quantization and Training of NNs for Efficient Integer-Arithmetic-Only Inference</i> , CVPR	8-bit integer quantization of weights and activations with a quantization-aware training scheme	ImageNet, COCO (MobileNets on ARM CPUs)	Improved accuracy-latency trade-off over float models on mobile CPUs	Basis of our post-training quantization experiments
8	Kong et al. (2020), <i>PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition</i> , IEEE/ACM TASLP	CNN14 and related CNN families pretrained on AudioSet, transferred to downstream tasks	AudioSet; transfer to ESC-50 and others	0.439 mAP on AudioSet at publication; pretrained CNNs transfer strongly to small datasets	Evidence that pretraining matters for CNNs too, so a fair CNN-vs-AST comparison must account for it
9	Park et al. (2019), <i>SpecAugment</i> , Interspeech	Time warping, frequency masking and time masking applied directly on the spectrogram	LibriSpeech, Switchboard	State-of-the-art ASR results at publication from augmentation alone	Cheap spectrogram-level augmentation we apply identically to all models
10	Han, Mao & Dally (2016), <i>Deep Compression</i> , ICLR	Magnitude pruning, weight quantization and Huffman coding pipeline	ImageNet (AlexNet, VGG-16), MNIST	35x (AlexNet) and 49x (VGG-16) size reduction without accuracy loss	Basis of our pruning experiments; also shows why size and latency must be measured separately

Synthesis. Papers 1–3 fix the benchmark and the CNN reference point (roughly 73–79% on UrbanSound8K). Paper 4 supports adding recurrence for temporal structure, although the polyphonic setting there differs from our single-label clips. Papers 5 and 8 show that pretrained models dominate accuracy, so the AST-versus-CNN comparison must separate the effect of architecture from the effect of pretraining; we therefore include a from-scratch and a pretrained run of the mobile-native CNN (Section 4.2) and report the pretraining status of every model. Papers 6, 7 and 10 supply the efficiency axis, and paper 9 supplies a shared augmentation. The gap we address is that these works report accuracy and efficiency separately, on different datasets; we measure both on one benchmark under one protocol.

3. Dataset Acquisition and Preprocessing Pipeline

Dataset. UrbanSound8K is obtained from the dataset authors (NYU). It contains 8,732 labelled WAV excerpts of at most 4 s in ten classes. The metadata file gives class, fold, salience and source ID for each clip. The class distribution is imbalanced: gun shot (374 clips), car horn (429) and siren (929) are smaller than the seven classes with 1,000 clips each. This is why macro-F1 is reported alongside accuracy. Most clips are exactly 4 s; shorter clips are zero-padded.

Pipeline (applied identically to every model).

- Load each clip as mono and resample to **16 kHz**. 16 kHz is chosen because the pretrained AST checkpoint expects it, so one resampled copy of the corpus serves all three models.
- Zero-pad or crop to a fixed **4.0 s** window (64,000 samples).

- **CNN and CRNN input:** log-mel spectrogram (power to dB) with 128 mel bands, FFT size 1024, hop 320 samples (20 ms), 20–8000 Hz, giving a 128 x 201 (mel x frames) input.
- **AST input:** the standard AST filter-bank front-end (128 mel bins, 10 ms shift, 400 frames for 4 s) computed from the same 16 kHz waveforms, as required by the pretrained checkpoint. This is the one place the front-end differs and it is reported as such.
- **Normalisation:** mean and standard deviation computed on the training folds only, in each cross-validation round, to prevent leakage. AST uses the checkpoint's own normalisation constants.
- **Augmentation (training batches only):** SpecAugment-style masking, two frequency masks of up to 16 bins and two time masks of up to 20 frames. No waveform-level augmentation in the main comparison, so every model sees the same augmentation. Waveform augmentation (time-stretch, pitch-shift) is kept as an optional ablation only.
- **Caching:** the 16 kHz padded waveforms and the log-mel features are stored on disk once and shared by all members, so every model trains from identical inputs.

Parameter	Decision
Sample rate / clip length	16,000 Hz / 4.0 s (64,000 samples)
Log-mel (CNN, CRNN, MobileNetV3)	n_fft 1024, hop 320, 128 mels, 20–8000 Hz, output 128 x 201
AST front-end	128-bin filter bank, 10 ms shift, 400 frames
Split per round k	Test = fold k; validation = fold (k mod 10) + 1; train = remaining 8 folds; k = 1 ... 10
Model selection	Best validation-accuracy epoch; the test fold is touched once per round
Reported numbers	Mean $\pm$ standard deviation over the 10 test folds

4. Models, Common Protocol and Preliminary Results

4.1 Common training protocol

The following settings are fixed for every model unless Section 4.2–4.4 states otherwise:

Item	Decision
Loss / optimiser	Cross-entropy with label smoothing 0.1; AdamW, weight decay 0.01
Learning rate	1e-3 for from-scratch models, with 2 warm-up epochs then cosine decay
Batch size / epochs	64 / up to 40, early stopping on validation accuracy with patience 8
Precision / seed	Mixed precision; seed 42 for all runs (additional seeds only if time permits)
Metrics	Accuracy, macro-F1, per-class confusion matrix (summed over folds)

4.2 Model 1 — CNN baseline and mobile-native variant (Rohan Sinha)

CNN baseline. Four convolutional blocks (3x3 convolution, batch normalisation, ReLU, 2x2 max pooling) with 32, 64, 128 and 256 channels, followed by global average pooling, dropout 0.3 and a 10-way linear layer. It is trained from scratch under the common protocol. It is the deliberately simple reference point, in the spirit of papers [2] and [3].

Mobile-native variant. MobileNetV3-Small, chosen over the Large variant because it is the smaller edge-oriented point of paper [6]. The single-channel spectrogram is fed in by summing the first-layer RGB filters, and the classifier head is replaced by a 10-way layer. Two runs are made: (a) from scratch, and (b) initialised from ImageNet weights with learning rate 3e-4. Run (b) is the pretraining control that makes the comparison with the pretrained AST fair. If MobileNetV3-Small underperforms badly, EfficientNet-Lite0 replaces it; this is decided only by validation accuracy, never by test-fold results.

Shared tooling owned by this member: the preprocessing pipeline (Section 3) and the efficiency-benchmarking harness (Section 4.6).

4.3 Model 2 — CRNN (Akshdeep Singh)

Three convolutional blocks (3x3 convolution, batch normalisation, ReLU) with 32, 64 and 128 channels and pooling of (2,2), (2,2) and (2,1) over (frequency, time). This shrinks the frequency axis from 128 to 16 while keeping 50 time steps. The remaining frequency and channel dimensions are flattened into one 2,048-dimensional vector per time step, projected to 128 dimensions, and passed to a single-layer bidirectional GRU with hidden size 128. Mean and max pooling over time are concatenated, followed by dropout 0.3 and a 10-way linear layer. A GRU is used rather than an LSTM because it has fewer parameters, which matters for the efficiency goal; an LSTM version is an optional ablation. Trained from scratch under the common protocol.

4.4 Model 3 — Audio Spectrogram Transformer (Siddharth Nair)

The pretrained AST of paper [5] (ImageNet and AudioSet initialisation, from the authors' public code release) with its input length set to 400 frames and the head replaced by a 10-way layer. Fine-tuning settings: AdamW, learning rate 5e-5 with cosine decay, batch size 16, mixed precision, 5 epochs per fold, SpecAugment masking as in Section 3. All 10 folds are run. If by 14 Oct the remaining GPU time cannot cover 10 folds, the fallback is a fixed subset of folds 1, 5 and 9 for AST and its compressed variant, with the full 10 folds kept for the other models and the subset stated clearly in the results table.

4.5 Preliminary results

Results are from the official 10-fold protocol (mean $\pm$ std over folds); the highlighted cells are filled from the training runs and benchmark harness, and are not estimated.

Model	Pretrained	Accuracy (%)	Macro-F1	Params (M)	Size (MB)	CPU latency (ms)
CNN baseline	No	[run]	[run]	[run]	[run]	[run]
MobileNetV3-Small	No	[run]	[run]	[run]	[run]	[run]
MobileNetV3-Small	ImageNet	[run]	[run]	[run]	[run]	[run]
CRNN (CNN + BiGRU)	No	[run]	[run]	[run]	[run]	[run]
AST	ImageNet + AudioSet	pending	pending	pending	pending	pending

Expected outcomes (hypotheses, not measured results). Before the runs are complete, the team's expectations are stated below so the final results can be compared against them. They are derived from published results on this benchmark (papers [2], [3], [5], [6]) and from parameter and FLOP counts of the specified architectures. Parameter and size figures follow directly from the designs; accuracy and latency ranges are the least certain and will be replaced by the measured values above.

Model	Pretrained	Accuracy (%)	Macro-F1	Params (M)	Size (MB, fp32)	CPU latency, 1 thread (ms)
CNN baseline	No	75–82	0.74–0.81	~0.39	~1.6	5–15
MobileNetV3-Small	No	70–80	0.69–0.79	~1.5	~6	3–10
MobileNetV3-Small	ImageNet	80–87	0.79–0.86	~1.5	~6	3–10
CRNN (CNN + BiGRU)	No	76–84	0.75–0.83	~0.56	~2.2	5–20
AST	ImageNet + AudioSet	90–95	0.89–0.95	~87	~340	500–1500

Expected ordering: AST is the most accurate but roughly 50–100 times slower and about 200 times larger than the small models; pretraining should lift the mobile-native model by several points; 8-bit quantization should cut size by about 3–4 times with a small accuracy loss on the CNNs, while AST is the model most at risk of larger degradation. Latency depends on the benchmarking machine, so the ratios between models matter more than the absolute values.

Analysis to be written from the results: which classes each model confuses (air conditioner versus engine idling, and drilling versus jackhammer, are the acoustically closest pairs), whether the CRNN's recurrence helps on 4 s clips, and how much accuracy the mobile-native model gives up for its savings in size and latency. One confusion matrix per model and the per-fold table accompany the results.

4.6 Efficiency benchmarking and compression protocol (fixed)

Item	Decision
Parameters / size	Total trainable parameters; size is the saved model file in MB (fp32, int8 and sparse variants reported separately)
Latency	Batch size 1, one 4 s clip, CPU only. 20 warm-up and 200 timed runs; report median and 95th percentile at 1 thread and at 4 threads. All models timed on one machine (the one running the harness). Feature extraction is timed separately from the forward pass.
Quantization	Post-training 8-bit: dynamic quantization for linear and recurrent layers (CRNN, AST); static quantization with 100 calibration clips from the training folds for the convolutional models. Evaluated on all folds with no retraining.
Pruning	Global magnitude (L1) unstructured pruning at 30%, 50% and 70% sparsity with 3 epochs of fine-tuning afterwards; evaluated on folds 1, 5 and 9 for every model. Reports non-zero parameters and compressed (gzip) size, and notes that unstructured sparsity does not by itself reduce CPU latency.
On-device mobile measurement	Out of scope. Efficiency is reported on CPU only and this is stated as a limitation in the final report.

5. Individual Contribution Log (to date)

Each member owns one model end-to-end and is individually responsible for defending it in the viva. Shared work is split as shown.

Task	Rohan Sinha	Akshdeep Singh	Siddharth Nair
Dataset download, EDA, class statistics	Owner	Review	Review
Shared preprocessing pipeline and feature cache	Owner	Uses	Uses
CNN baseline + MobileNetV3 variants	Owner	—	—
Efficiency harness (params, size, latency)	Owner	Uses	Uses
CRNN implementation and tuning	—	Owner	—
AST fine-tuning and quantization / pruning study	—	—	Owner
Literature table (papers 1, 2, 3, 6 / 4, 8, 9 / 5, 7, 10)	Papers 1, 2, 3, 6	Papers 4, 8, 9	Papers 5, 7, 10
Interim report writing	Sections 1, 3, 4.1–4.2, 4.6	Sections 4.3, 5	Sections 2, 4.4, 6
Final slides and presentation	Own model + results	Own model + error analysis	Own model + compression

Version control. A single shared GitHub repository is created by the team, with one branch per model (CNN, CRNN, AST) and one shared branch for preprocessing and the benchmarking harness. Each member commits their own work from their own account at regular intervals, not in a single push, so that the commit history can be used as evidence of individual contribution.

Use of external tools (academic-integrity declaration). Code and text are the team's own work except where cited. The literature summary table and the text of this interim report were drafted with an LLM (Claude, Anthropic), and the members verified the cited results against the original papers before submission. Any later use of an LLM for code, experiments or the final report will be listed in the final report with the exact place of use.

Task matrix confirmed and signed by all members:

Rohan Sinha
Date:

Akshdeep Singh
Date:

Siddharth Nair
Date:

6. Risks and Plan for Remaining Work

Risk	Likelihood / impact	Mitigation
AST fine-tuning is slow or exceeds available GPU memory (about 87M parameters)	Medium / High	Mixed precision, batch size 16 with gradient accumulation if needed, 5 epochs per fold; fallback to folds 1, 5 and 9 decided on 14 Oct (Section 4.4)
Unfair comparison: AST is pretrained, CNN and CRNN are trained from scratch	High / Medium	Pretraining status is a column in the results table; ImageNet-pretrained MobileNetV3-Small is included as a control (Section 4.2)
Fold leakage or inconsistent normalisation across members	Low / High	One shared split rule and one shared feature cache (Section 3); no member re-splits the data
Quantization or pruning degrades the transformer sharply	Medium / Medium	Report it as a finding; dynamic 8-bit quantization first; pruning sweep over three sparsity levels
Latency numbers not comparable across machines	Medium / Medium	All models timed on one machine with fixed thread counts (Section 4.6)
Repository not yet in place, so commit evidence starts late	High / Medium	Repository created and the preprocessing pipeline pushed first; every member commits from their own account from then on
Limited working days around the mid-term exam period	High / Medium	Fixed weekly work slots; heavy GPU runs scheduled in blocks; AST setup done in parallel with CNN/CRNN runs

Timeline to final submission

Milestone	Owner	Target
Repository created; preprocessing pipeline and feature cache pushed	Rohan Sinha	03 Oct 2026
CNN, MobileNetV3 and CRNN 10-fold results	Rohan Sinha, Akshdeep Singh	09 Oct 2026
AST fine-tuned (all folds or fallback subset); tuning complete	Siddharth Nair	14 Oct 2026
Quantization and pruning for all models; size/latency benchmark	Siddharth Nair (compression), Rohan Sinha (harness)	21 Oct 2026
Comparison table, error analysis, confusion matrices	All members	24 Oct 2026
Slides and presentation rehearsal	All members	25 Oct 2026
Final presentations	All members	26–29 Oct 2026
Final report, code repo, Turnitin check (plagiarism $\leq 15\%$, AI $\leq 20\%$), submission	All members	31 Oct 2026 (hard deadline)

References

- [1] J. Salamon, C. Jacoby, and J. P. Bello, “A dataset and taxonomy for urban sound research,” in *Proc. 22nd ACM Int. Conf. Multimedia (ACM-MM)*, 2014, pp. 1041–1044.
- [2] J. Salamon and J. P. Bello, “Deep convolutional neural networks and data augmentation for environmental sound classification,” *IEEE Signal Process. Lett.*, vol. 24, no. 3, pp. 279–283, 2017.
- [3] K. J. Piczak, “Environmental sound classification with convolutional neural networks,” in *IEEE Int. Workshop Mach. Learn. Signal Process. (MLSP)*, 2015, pp. 1–6.
- [4] Y. Gong, Y.-A. Chung, and J. Glass, “AST: Audio spectrogram transformer,” in *Proc. Interspeech*, 2021, pp. 571–575.
- [5] E. Çakır, G. Parascandolo, T. Heittola, H. Huttunen, and T. Virtanen, “Convolutional recurrent neural networks for polyphonic sound event detection,” *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 25, no. 6, pp. 1291–1303, 2017.
- [6] A. Howard, M. Sandler, G. Chu, et al., “Searching for MobileNetV3,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1314–1324.
- [7] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.
- [8] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, “PANNS: Large-scale pretrained audio neural networks for audio pattern recognition,” *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 28, pp. 2880–2894, 2020.
- [9] D. S. Park, W. Chan, Y. Zhang, et al., “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *Proc. Interspeech*, 2019, pp. 2613–2617.
- [10] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.